

01 / BridgeRemit technical report

Project scope

BridgeRemit is a coursework DApp for migrant-worker family remittances and SME payroll or supplier payments. It combines a responsive English-language interface, wallet signatures, four interacting Solidity contracts, a Python Flask REST API, and persistent storage. Render is the intended host for the web application and Postgres database; the financial state belongs to a separately deployed Ethereum test network.

Current result

Four Solidity contracts and seven deployment/configuration transactions are verified on Sepolia. The backend has been migrated to Python Flask / Web3.py. Sixteen Solidity tests and nine Flask integration tests passed on 5 October 2026. Flask tests use actual Ganache transactions for creation, claim and audit, with SQLite reopening, signature replay protection and forwarding receipts. These are local tests, not a completed public multi-wallet remittance demonstration.

Financial boundary

The only transferable asset is a six-decimal test token, dUSD. It is not USDC, is not backed by dollars and has no redemption facility. Destination currency estimates are indicative values; recipients actually receive dUSD on the same chain. There is no cross-chain bridge, real cash-in, bank cash-out or production identity verification. The system demonstrates a cross-border business workflow, rather than a licensed remittance service.

Submission use

Source, tests, configuration, architecture diagrams, user manual and a ten-minute demonstration plan accompany this report. The GitHub repository should preserve real contribution history. The team should attach public URLs, contract addresses, transaction hashes and feedback records after executing those steps. A working local prototype should not be labelled enterprise-ready or production-validated without the additional security, operational and compliance work described here.

02 / Problem and stakeholder analysis

Evidence

The World Bank Remittance Prices Worldwide Q3 2025 report records a global average remittance cost of 6.36%, compared with 6.49% in Q1 2025. This is evidence that remittance costs remain material across surveyed corridors. It is not a Singapore-to-Philippines quote, an SME trade-finance benchmark or proof of the prototype's savings. Source: World Bank RPW Issue 54 / Q3 2025, published at the report URL in the references.

Target users

A migrant worker may need to send a modest household transfer and understand how much the recipient receives. An SME finance operator may need to pay multiple employees or suppliers and reconcile payment status. A recipient needs a clear claim and dispute process. Administrators review test eligibility and manage corridors; auditors inspect records; settlement agents may register an explicitly simulated off-ramp receipt.

Pain points and hypotheses

The proposed pain points are uncertainty about fees and completion status, repeated administrative effort for multiple payments, and reliance on one provider's private ledger for reconciliation. These are design hypotheses for the selected users, not interview findings. A blockchain workflow can make token custody and release independently verifiable. A conventional centralized database could provide a simpler user experience and reversal process; blockchain is justified only where shared verification and deterministic custody are valuable.

Validation plan

Recruit consenting participants from both target groups or run clearly labelled scenario evaluation. Ask users to identify the actual receiving asset, inspect total quoted fees, submit a test payment and recover from a rejected wallet signature without duplicate payment. Record task completion, time, errors and feedback. Do not collect real identity documents or bank details. No stakeholder response or satisfaction score is asserted in this draft.

03 / Requirements and success indicators

Functional scope

The course requires at least three roles, MetaMask signing, at least three interacting contracts and ten blockchain transaction types. BridgeRemit has customer, auditor, settlement-agent and administrator roles. Its Registry, DemoUSD, CorridorBook and Escrow interact through immutable addresses. Eighteen public methods change chain state; sixteen method categories have direct interface controls, while transferFrom is used by escrow and token transfer remains directly callable.

Business functions

A customer can register, obtain approval, request test tokens, authorize escrow, create a payment or an atomic batch, and cancel or request an expired refund. A recipient claims before the deadline. Either participant may dispute a pending payment before expiry. An administrator resolves a dispute, changes corridors and fees, pauses new business and withdraws earned fees. An approved agent can attest once to a completed payment.

Nonfunctional requirements

Security requirements include wallet-bound sessions, role enforcement on chain, parameterized database queries, secret-free cloud signing, replay protection and controlled request sizes. Usability requires clear receiving-asset labels, a fee estimate before signing and explicit failure recovery. Responsive layout supports smaller screens. Durability is provided by Postgres on Render, not the web instance filesystem. Local storage uses SQLite for reproducible development.

Measurable acceptance

Local correctness criteria are rejection of unauthorized mutation, nonce replay, duplicate payment references and double release; all-or-nothing batches; and agreement between pending liabilities, earned fees and escrow balances under normal operation. Future usability targets should be defined before observation rather than invented afterward. Complete economic evaluation must add on-ramp costs, spreads, payer and recipient gas, platform fees and off-ramp costs. The example 0.5% fee is a parameter, not a measured end-to-end saving.

04 / System architecture and data ownership

Deployment layers

The browser loads static HTML, CSS and a locally served ethers library from the Flask service. It accesses the same-origin API for authentication, invoices and verified receipts. MetaMask signs user messages and sends transactions directly through its provider. The API uses a read-only RPC connection to inspect contract roles, payment state and receipts. The backend never needs a user signing key.

On-chain records

Registry stores roles and approval flags. DemoUSD stores balances and allowances. CorridorBook stores rate, fee basis points and active status. Escrow stores sender, recipient, gross amount, platform fee, indicative destination amount, captured rate, corridor ID, random reference hash, expiry, status and optional attestation hash. Funds are transferred by the test token, not by the database.

Off-chain records

The database stores challenges, hashed sessions, invoice drafts and verified transaction receipts. Invoice metadata contains a UUID, owner wallet, recipient wallet, amount, memo and creation time. The random UUID hash links the draft to an on-chain reference without placing the memo on chain. Wallet ownership and role are read from the contract; there is no SQL wallet-role table that users can modify to elevate themselves.

Consistency model

The chain is authoritative for funds. Saving a database draft does not create a payment. A successful chain transaction followed by a failed receipt-save request remains successful on chain; the interface tells the user to refresh rather than resubmit funds. The API audit table is a collection of submitted, verified receipts, not a full chain index. Production reconciliation would require a persistent block cursor, reorganization handling and automatic replay of confirmed logs. Five editable Mermaid diagrams accompany this report in ARCHITECTURE.md.

05 / Smart contract responsibilities

ParticipantRegistry

The deployment wallet is the immutable administrator. Unregistered wallets can only self-register as unapproved customers. Only the administrator assigns customer, auditor or agent roles and changes approval. The administrator cannot be reassigned through setRole. Approval acts as a simulated business gate rather than evidence of a real KYC process. New payments require both sender and recipient to be approved customers.

DemoUSD and CorridorBook

The token has six decimals and provides approve, transfer and transferFrom. Its faucet mints 10,000 dUSD to an approved customer at most once per day. There is no reserve backing or production monetary policy. CorridorBook is administered through Registry authority. Rates are stored with six decimals; fees cannot exceed 500 basis points. The configured SG-PH, SG-IN and SG-BD rates are fabricated demonstration inputs, not current market FX.

RemittanceEscrow

The escrow receives token custody through transferFrom. It requires a nonzero unused reference hash, distinct approved counterparties, amount from 1 to 100,000 dUSD, an active corridor and expiry between five minutes and thirty days. A caller supplies the maximum acceptable fee and minimum acceptable indicative destination amount. If the administrative quote changes unfavorably before transaction execution, the call reverts.

Contract interaction and limits

Batch creation invokes the same checks for each row and permits at most twenty rows. A single failure reverts all token deposits and reference reservations in the batch. Existing payments capture their fee and rate, so future corridor changes do not rewrite their economics. The escrow token, registry and book addresses are immutable. A future migration to real external tokens would require additional integration and security analysis rather than merely replacing an address.

06 / Payment lifecycle and custody invariant

Normal completion

Creation locks the gross deposit in Pending. Before expiry, the designated recipient calls claim. The escrow first changes status to Completed, adds the fee to earnedFees, then transfers amount minus fee to the recipient. These changes are atomic. A second claim is rejected. Platform fees accrue only on completion, including administrator-authorized dispute release.

Cancellation and expiry

The sender can cancel any still-pending payment and recover its full gross amount. A pending payment cannot be claimed after its deadline; the sender may then call refundExpired. There is no fee charge for cancelled or refunded funds. Cancellation is an explicit product policy: the recipient should not treat an unclaimed pending payment as irreversible settlement. Competing claim and cancel transactions are ordered by chain execution, and only the first permitted state transition succeeds.

Dispute handling

Before expiry, either sender or recipient can mark a pending payment Disputed. Claim, cancellation and expiry refund then stop. Only the administrator resolves the case by releasing net funds and fee, or by refunding the full sender deposit. An expired disputed payment still cannot auto-refund. This avoids two independent payout paths, but places dispute availability and fairness under administrator control. Lost administrator access can lock disputed funds.

Conservation

Under normal contract operation, escrow token balance equals the gross amount of all Pending and Disputed payments plus earnedFees. Withdrawal zeroes earnedFees before transferring only that amount to the administrator. Direct external token donations can increase escrow balance; a general production invariant therefore uses balance greater than or equal to liabilities. Pausing affects only creation. Revoking a participant's eligibility does not remove exit rights on existing deposits.

07 / Wallet authentication and authorization

Sign-in protocol

The backend creates a challenge bound to the application origin, wallet address, chain ID, a random nonce and a five-minute expiry. The challenge explicitly states that it signs the user in and does not transfer funds. The browser asks the connected wallet to sign it. The server recovers the signer and atomically deletes the matching nonce before issuing a one-hour session. Replaying the same signature cannot consume the nonce twice.

Session handling

The random session bearer value is sent only in a cookie. Its SHA-256 hash is stored in the database. Production cookies use Secure, HttpOnly and SameSite=Strict. The write API rejects mismatched Origin values and non-JSON requests. Logout deletes the session, and expired challenges and sessions are periodically removed. This implementation supports ordinary externally owned accounts, not ERC-1271 smart-contract wallet signatures.

Role enforcement

Roles are read from the live Registry contract. Invoice creation requires an approved customer. Customers read only their own invoice drafts, while auditor and administrator roles can read the verified receipt audit feed. Contract write restrictions remain authoritative even if a user edits browser controls. The UI connects each contract to the user's signer; no backend administrator endpoint signs a payment for the user.

Operational limits

Request counters are currently maintained in process memory and use the direct socket address. This is conservative behind a shared proxy and is not a distributed rate-limit service. A larger deployment needs trusted proxy configuration, shared counters, abuse monitoring and RPC quotas. Ethereum addresses, transactions and their timing are public and linkable. Hashing an invoice identifier is not a guarantee of anonymity.

08 / Frontend workflow and user experience

Dashboard

The interface presents test balance, sent amount in the loaded history window, pending count and role. A form shows route, amount, recipient, memo and expiry. The quote panel separates indicative destination currency, platform fee, actual dUSD receipt and estimated ETH gas. The corridor chart shows amounts from currently loaded records and is not labelled as all-time analytics. The sample-mode banner states that all displayed balances and payments are fictional.

Transaction confirmation

Before sending a contract transaction, the interface calls `estimateGas` and displays a confirmation dialog with the operation and estimated fee. MetaMask then supplies the final transaction review and signature. Public Sepolia flows wait for two confirmations; local flows wait for one. These policies are convenient confirmation rules rather than guarantees of finality. After success, decoded receipt events are displayed and the receipt is submitted for backend verification.

Recovery and batching

A cancelled wallet signature can leave a stored invoice draft. The history view can retry that same random reference; the on-chain used-reference check prevents the draft from creating another transfer after success. A batch consists of one to twenty address, amount and memo rows. All rows are validated before signature, then the contract enforces atomic execution. Off-chain drafts left by a failed batch remain drafts, not evidence of partial payment.

Accessibility and verification

The page has labelled inputs, semantic buttons, responsive grid layouts and an aria-live status region. Data inserted into tables is HTML-escaped. Numeric input is checked for six decimal places and bounded amounts. Real MetaMask signing, assistive-technology testing, localization interviews and mobile-wallet integration remain acceptance tasks; local browser inspection alone does not establish those outcomes.

09 / REST API and persistent data

Endpoints

GET /health checks database availability. GET /api/config returns only public deployment settings. GET /api/deploy-artifacts serves public ABI and bytecode for the browser deployment assistant. POST /api/auth/challenge and /verify handle signatures; /logout revokes sessions. GET /api/snapshot reads wallet-scoped chain state. GET and POST /api/invoices manage drafts. POST /api/receipts verifies a transaction and GET /api/audit exposes the role-restricted receipt feed.

Storage implementation

Development uses Python sqlite3 with write-ahead logging; Flask tests use temporary persistent SQLite files and verify reopening preserves drafts and sessions. Production requires DATABASE_URL and uses psycopg connections scoped to individual transactions. Missing production storage fails startup. The original PostgreSQL schema is retained, with parameterized SQL, UUIDs, BIGINT timestamps and unique transaction hashes.

Receipt verification

A receipt must be successful and its transaction sender must match the signed-in wallet. Its target must be a configured contract, or a configured contract must have emitted a known event in the receipt. This supports wallet forwarding transactions while retaining sender checks. Confirmations are required, gas comes from the receipt, and canonical transaction hashes make duplicate submissions idempotent. No receipt proves a real bank payout.

Scale and failure behavior

Historical snapshots read windows of at most one hundred IDs, filtering by role and counterparties, with a next-offset for older windows. Serial RPC reads are sufficient for a coursework dataset but will be slow at scale. Production should index events and query a wallet-indexed read model. Body size is bounded, errors use consistent JSON responses and static paths are checked against the public root. RPC failure should interrupt reads safely without creating a second transfer.

10 / Security evaluation and residual risks

Verified controls

Local tests reject unauthorized role assignment, corridor updates, claims, dispute resolutions, agent attestations and fee withdrawals. They verify faucet approval and cooldown, reference replay rejection, quote slippage protection, correct gross refunds, claim expiry, dispute freezes and atomic batch rollback. The final balance test compares remaining deposits with the escrow token balance after fee withdrawal.

Web controls

The API uses parameterized statements and per-wallet ownership filters. Signatures are validated server-side and nonces are consumed atomically. Requests have rate and body limits. Responses set Content-Security-Policy, X-Content-Type-Options, X-Frame-Options and a restrictive referrer policy. Client dependencies are served locally. Sessions use cookie protections rather than localStorage bearer tokens.

Unverified or missing assurance

These controls are not a formal smart-contract audit, penetration test or availability assessment. No fuzz campaign, model checker, independent audit, mainnet validation, distributed load test, Postgres disaster-recovery exercise or browser-extension threat test has been performed. The controlled test token has no callbacks; replacing it with arbitrary tokens changes the reentrancy and balance assumptions.

Governance and adoption

A single administrator controls role approval, reference rates and dispute decisions. The code has no administrator rotation, multisignature governance or emergency recovery. A production financial application would require licensing analysis for the actual jurisdictions, compliant KYC/AML processes, custody and redemption policies, dispute governance, user support, privacy review, incident response and independent security assurance. No legal conclusion or production fitness is asserted here.

11 / Testing results and gas analysis

Executed tests

The final local run completed twenty-five tests with zero failures: sixteen contract tests and nine Flask API tests. Contract cases cover authority boundaries, registration, faucet restrictions, settlement, replay, gross refund, eligibility and allowance, dispute freeze and resolution, expiry, pause behavior, quote protection, batch rollback, batch success, batch/input bounds, attestation and conservation. API cases cover health/config, origin and address validation, wallet nonce replay, chain-derived roles, invoice isolation, audit access, receipt ownership/idempotence, logout and static traversal.

Gas measurement method

The project records gasUsed from successful Ganache receipts using the compiled optimizer settings. Values are the last executed fixture for each operation, not averages over a representative dataset. Cold/warm storage, recipient balance, role changes and existing state affect the numbers. Network ETH price and token fiat price are not estimated from these local values. The report table below records representative values from the completed run.

Interpretation

The recorded two-payment batch used fewer gas per payment than the recorded single-create fixture, but the fixtures have different state access patterns. This is a useful optimization illustration, not a controlled benchmark or a guarantee of savings for every batch. All runtime bytecodes are below the EVM contract-size cap in this build. Batch size and amount bounds limit execution risk, while viaIR and optimizer settings reduce code size.

Remaining validation

Actual multi-wallet Sepolia remittance and claim, cloud database restart persistence, user task timing and end-to-end remittance economics remain unmeasured. Public deployment evidence is in DEPLOYMENT_STATUS.md and SEPOLIA_VERIFICATION.json; Flask migration evidence is in FLASK_MIGRATION.md. Passing local tests do not establish public throughput, real costs or user satisfaction.

Operation	Recorded gasUsed
create	302790
batchCreate2	545969
claim	57345
dispute	36112
resolveRefund	53036
refundExpired	49731
attest	61672

12 / Deployment acceptance and references

Render configuration

render.yaml defines a free Python Flask web service and a separate free Postgres instance. It installs from the committed lockfile, compiles contracts and copies the local browser library, then starts the server on Render's PORT. APP_ORIGIN must match the final HTTPS service URL exactly. DATABASE_URL is injected from Postgres. RPC_URL connects to Sepolia, and deployment.sepolia.json contains public contract configuration. No user or deployment signing key is required on Render.

Public contract deployment

The local deployment script and browser deployment assistant enforce Sepolia chain ID 11155111. The assistant asks the wallet to deploy Registry, DemoUSD, CorridorBook and Escrow, followed by three corridor-setting transactions. The wallet becomes administrator. Its output contains addresses, ABI and transaction hashes without private keys; the complete public configuration must be committed before the backend can use the contracts. A healthy viewer-only website with chainConfigured=false is not a complete public DApp acceptance result.

Acceptance and project management

Record the public URL, deployed contracts and deployment transactions. Execute sender and recipient flows with actual MetaMask, check audit isolation, restart the web service and confirm persisted drafts and receipts. Confirm the account's free-plan eligibility and database expiry before a demonstration. Maintain honest GitHub commits, assign report sections and capture real stakeholder evidence. The accompanying contribution template and demonstration script identify items that must be supplied by the team.

References

World Bank, Remittance Prices Worldwide Q3 2025: https://remittanceprices.worldbank.org/sites/default/files/2026-04/RPW_main_report_and_annex_Q325.pdf .
Render, native runtimes: <https://render.com/docs/native-runtimes> . Render, Blueprint specification: <https://render.com/docs/blueprint-spec> . Render, free-plan restrictions: <https://render.com/docs/free> .
Ethereum, development networks: <https://ethereum.org/developers/docs/networks/> . Course source: SC6113 Group Project.docx supplied by the user. Implementation evidence: contracts/BridgeRemit.sol, tests/*.test.mjs and the local test output dated 5 October 2026.